

QUESTIONS

Q1

Banking Transaction System –
Exception Handling
10 marks

Q2

Hospital Patient Registration –
User-Defined Exception
10 marks

Q3

E-Commerce Order Processing
– Built-in Exceptions
10 marks

Q4

Hospital Emergency System –
Thread Priority
10 marks

Q5

Online Food Delivery – Multiple
Threads and Execution Order
10 marks

5/5 answered

Q1 Banking Transaction System – Exception Handling

10 marks

Develop a Java program for an ATM withdrawal system. The program accepts the account balance and withdrawal amount. The program must:

- Accept the current account balance and withdrawal amount.
- Throw a user-defined `InsufficientBalanceException` when the withdrawal amount exceeds the balance.
- Handle withdrawal amounts that are zero or negative.
- Handle invalid numeric input using a suitable built-in exception.
- Display a meaningful transaction result.
- Use try-catch-finally so that the transaction completion message is always displayed.

Sample Test Cases:

Input / Scenario Expected Result Purpose

10000 3000 Normal case

10000 12000 Boundary / exception case

10000 -500 Invalid input

abc 3000 Invalid input

Your Code

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String args[]){
4         Scanner sc=new Scanner(System.in);
5         try{
6             double bal=sc.nextDouble();
7             double wd=sc.nextDouble();
8             if(wd<=0){
9                 throw new IllegalArgumentException("Withdrawal amount must be greater than 0")
10             }
11         }
12     }
13 }
```

Input

10000 3000

Output

```
Withdraw Successfull
Remaining balance : 7000.0
Transaction completed.
```

10 marks

5/5 answered

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String args[]){
4         Scanner sc=new Scanner(System.in);
5         try{
6             double bal=sc.nextDouble();
7             double wd=sc.nextDouble();
8             if(wd<=0){
9                 throw new IllegalArgumentException("Withdrawal amount n
10            }
11            if(wd>bal){
12                throw new InsufficientBalanceException("Insufficient ba
13            }
14            bal=bal-wd;
15            System.out.println("Withdraw Successfull");
16            System.out.println("Remaining balance : "+bal);
17        }catch(InsufficientBalanceException e){
18            System.out.println(e.getMessage());
19        }catch(IllegalArgumentException e){
20            System.out.println("Invalid input : "+e.getMessage());
21        }catch(Exception e){
22            System.out.println("Invalid Numeric Input. ");
23        }
24        finally{
25            System.out.println("Transaction completed. ");
26        }
27        sc.close();
28    }
29 }
30 class InsufficientBalanceException extends Exception{
31     public InsufficientBalanceException(String message){
32         super(message);
33     }
34 }
```

10000 3000

Output

```
Withdraw Successfull
Remaining balance : 7000.0
Transaction completed.
```


Q1 ✓
Banking Transaction System –
Exception Handling
10 marks

Q2 ✓
Hospital Patient Registration –
User-Defined Exception
10 marks

Q3 ✓
E-Commerce Order Processing
– Built-in Exceptions
10 marks

Q4 ✓
Hospital Emergency System –
Thread Priority
10 marks

Q5 ✓
Online Food Delivery – Multiple
Threads and Execution Order
10 marks

5/5 answered

Q2 Hospital Patient Registration – User-Defined Exception

10 marks

Develop a Java program for a hospital patient registration system. The program accepts the patient's name, age, and body temperature. Create suitable user-defined exception classes for invalid age and invalid temperature. Validate the input before registration. Use try-catch-finally and display a registration completion message from finally.

Sample Test Cases:

Input / Scenario Expected Result Purpose

Arun, 35, 98.6 Patient registered successfully Normal case

Meena, 12, 98.4 InvalidAgeException Boundary / exception case

Ravi, 45, 105.2 InvalidTemperatureException Invalid input

Kumar, -2, 98.1 InvalidAgeException Invalid input

Your Code

```
1 import java.util.Scanner;  
2 public class Main{  
3     public static void main(String args[]){  
4         Scanner sc=new Scanner(System.in);  
5         try{  
6             String name=sc.next();  
7             int age=sc.nextInt();  
8             double temp=sc.nextDouble();  
9             if(age<18){  
10                throw new InvalidAgeException();  
11            }  
12            if(temp<95 || temp>100){  
13                throw new InvalidTemperatureException();  
14            }  
15            System.out.println("Patient registered successfully. ");  
16        }catch(InvalidAgeException e){  
17            System.out.println("InvalidAgeException");  
18        }catch(InvalidTemperatureException e){
```

Input

Arun 35 98.6

Output

```
Patient registered successfully.  
Registration completed.
```


SIMATS Online Programming Lab

JAVA PROGRAMMING

```
7      int age=sc.nextInt();
8      double temp=sc.nextDouble();
9      if(age<18){
10         throw new InvalidAgeException();
11     }
12     if(temp<95 || temp>100){
13         throw new InvalidTemperatureException();
14     }
15     System.out.println("Patient registered successfully. ");
16 }catch(InvalidAgeException e){
17     System.out.println("InvalidAgeException");
18 }catch(InvalidTemperatureException e){
19     System.out.println("InvalidTemperatureException");
20 }finally{
21     System.out.println("Registration completed. ");
22 }
23 sc.close();
24 }
25 }
26 class InvalidAgeException extends Exception{
27     public InvalidAgeException(){
28         super();
29     }
30 }
31 class InvalidTemperatureException extends Exception{
32     public InvalidTemperatureException(){
33         super();
34     }
35 }
```


QUESTIONS

Q1 
Banking Transaction System –
Exception Handling
10 marks

Q2 
Hospital Patient Registration –
User-Defined Exception
10 marks

Q3 
E-Commerce Order Processing
– Built-in Exceptions
10 marks

Q4 
Hospital Emergency System –
Thread Priority
10 marks

Q5 
Online Food Delivery – Multiple
Threads and Execution Order
10 marks

5/5 answered

Q3 E-Commerce Order Processing – Built-in Exceptions

10 marks

Develop a Java program for an online shopping application. Store product names and prices in arrays and accept a product index and quantity from the customer.

The program must:

- Calculate total cost as quantity × price.
- Handle invalid product indices.
- Handle invalid numeric input.
- Reject zero or negative quantities.
- Use appropriate built-in exception classes.
- Continue execution after handling an invalid transaction.

Sample Test Cases:

Input / Scenario Expected Result Purpose

1, 2 Valid product; total cost calculated Normal case

5, 2 ArrayIndexOutOfBoundsException handled Boundary / exception case

2, -1 Invalid quantity Invalid input

abc, 2 InputMismatchException handled Invalid input

Your Code

```
1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3 public class Main{
4     public static void main(String args[]){
5         Scanner sc=new Scanner(System.in);
6         String prod[]={"Laptop","Phone","Headphones","Keyboard"};
7         double[] prices={50000,20000,2000,1500};
8         try{
9             int index=sc.nextInt();
10            int quan=sc.nextInt();
11            String pr=prod[index];
```

Input

1 2

Output

```
Product: Phone
Total cost: 40000.0
Transaction processing completed
```

Threads and Execution Order

10 marks

5/5 answered

Your Code

```
1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3 public class Main{
4     public static void main(String args[]){
5         Scanner sc=new Scanner(System.in);
6         String prod[]{"Laptop","Phone","Headphones","Keyboard"};
7         double[] prices={50000,20000,2000,1500};
8         try{
9             int index=sc.nextInt();
10            int quan=sc.nextInt();
11            String pr=prod[index];
12            if(quan<=0){
13                throw new IllegalArgumentException("Invalid quantity")
14            }
15            double t=prices[index]*quan;
16            System.out.println("Product: "+prod[index]);
17            System.out.println("Total cost: "+t);
18        }catch(ArrayIndexOutOfBoundsException e){
19            System.out.println("ArrayIndexOutOfBoundsException handled")
20        }catch(InputMismatchException e){
21            System.out.println("InputMismatchException handled");
22        }catch(IllegalArgumentException e){
23            System.out.println("Invalid quantity ,Invalid input");
24        }finally{
25            System.out.println("Transaction processing completed");
26        }
27        sc.close();
28    }
29 }
```

Submitted

QUESTIONS

Q1

Banking Transaction System –
Exception Handling
10 marks

Q2

Hospital Patient Registration –
User-Defined Exception
10 marks

Q3

E-Commerce Order Processing
– Built-in Exceptions
10 marks

Q4

Hospital Emergency System –
Thread Priority
10 marks

Q5

Online Food Delivery – Multiple
Threads and Execution Order
10 marks

5/5 answered

Q4 Hospital Emergency System – Thread Priority

10 marks

A hospital application creates threads for Emergency Patient Processing, General Patient Processing, and Billing Processing.

Develop a Java program that assigns different priorities to these threads. Run the program several times and analyze whether the emergency thread is guaranteed to execute first.

Then modify the design if the hospital requires a strict execution sequence rather than merely expressing scheduling preference.

Your answer must contain Java code and an execution analysis based on the actual thread behavior.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Emergency, General, Billing All threads execute; priority does not guarantee a fixed output order Normal execution

Emergency workload > General workload Emergency thread receives a higher priority but exact ordering is not guaranteed Concurrency / design

Strict emergency-before-billing requirement Use explicit coordination rather than relying only on priority Exception / boundary

Your Code

```
1 public class Main{
2     public static void main(String args[])throws InterruptedException{
3         System.out.println("==PRIORITY BASED EXECUTION==");
4         PatientThread emergency=new PatientThread("Emergency");
5         PatientThread general=new PatientThread("General");
6         PatientThread billing=new PatientThread("Billing");
7         emergency.setPriority(Thread.MAX_PRIORITY);
8         general.setPriority(Thread.NORM_PRIORITY);
9         billing.setPriority(Thread.MIN_PRIORITY);
10        emergency.start();
11        general.start();
12        billing.start();
13        emergency.join();
14    }
```

Input

stdin input

Output

```
==PRIORITY BASED EXECUTION==
Emergency Patient processing.
Billing Patient processing.
General Patient processing.
==STRICT EXECUTION SEQUENCE==
Emergency Patient processing
```

```

6      PatientThread billing=new PatientThread("Billing");
7      emergency.setPriority(Thread.MAX_PRIORITY);
8      general.setPriority(Thread.NORM_PRIORITY);
9      billing.setPriority(Thread.MIN_PRIORITY);
10     emergency.start();
11     general.start();
12     billing.start();
13     emergency.join();
14     general.join();
15     billing.join();
16     System.out.println("===STRICT EXECUTION SEQUENCE===");
17     PatientThread emergency2 = new PatientThread("Emergency");
18     PatientThread general2 =new PatientThread("General");
19     PatientThread billing2 = new PatientThread("Billing");
20     emergency2.start();
21     emergency2.join();
22     general2.start();
23     general2.join();
24     billing2.start();
25     billing2.join();
26     System.out.println("Strict sequence completed. ");
27 }
28 }
29 class PatientThread extends Thread{
30     String type;
31     PatientThread(String type){
32         this.type=type;
33     }
34     public void run(){
35         System.out.println(type+" Patient processing.");
36     }
37 }

```

Output

```

===PRIORITY BASED EXECUTION===
Emergency Patient processing.
Billing Patient processing.
General Patient processing.
===STRICT EXECUTION SEQUENCE===
Emergency Patient processing

```


QUESTIONS

Q1

Banking Transaction System –
Exception Handling
10 marks

Q2

Hospital Patient Registration –
User-Defined Exception
10 marks

Q3

E-Commerce Order Processing
– Built-in Exceptions
10 marks

Q4

Hospital Emergency System –
Thread Priority
10 marks

Q5

Online Food Delivery – Multiple
Threads and Execution Order
10 marks

5/5 answered

Q5 Online Food Delivery – Multiple Threads and Execution Order

10 marks

An online food-delivery application uses three threads: Order Processing, Payment Processing, and Delivery Assignment.

Develop a Java program in which these activities are represented by separate threads. Analyze the problem if all three threads are started independently.

Modify the program so that:

- Order processing completes before payment processing.
- Payment processing completes before delivery assignment.
- The program uses appropriate Java thread-management techniques.
- The output clearly shows the required execution order.

Explain the difference between simply starting all threads and explicitly controlling their execution order.

Sample Test Cases:

Input / Scenario Expected Result Analysis Focus

Order=Pizza; Payment=UPI; Delivery=Agent A Order → Payment → Delivery Normal execution

Order=Burger; Payment=Card; Delivery=Agent B Order → Payment → Delivery Concurrency / design

Payment=Failed Delivery assignment should not proceed Exception / boundary

Your Code

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String args[])throws InterruptedException{
4         Scanner sc=new Scanner(System.in);
5         String ord=sc.next();
6         String pm=sc.next();
7         String del=sc.next()+" "+sc.next();
8         System.out.println("Order: "+ord);
9         System.out.println("Payment: "+pm);
10        System.out.println("Delivery: "+del);
11    }
12 }
```

Input

pizza upi agent A

Output

Order: pizza
Payment: upi
Delivery: agent A

10 marks

5/5 answered

Your Code

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String args[]) throws InterruptedException{
4         Scanner sc=new Scanner(System.in);
5         String ord=sc.next();
6         String pm=sc.next();
7         String del=sc.next()+" "+sc.next();
8         System.out.println("Order: "+ord);
9         System.out.println("Payment: "+pm);
10        System.out.println("Delivery: "+del);
11        OrderThread ot=new OrderThread();
12        PaymentThread pt=new PaymentThread();
13        DeliveryThread dt=new DeliveryThread(del);
14        ot.start();
15        ot.join();
16        pt.start();
17        pt.join();
18        if(pm.equalsIgnoreCase("Failed")){
19            System.out.println("Payment failed , delivery assignment c
20        }else{
21            dt.start();
22            dt.join();
23        }
24        System.out.println("Execution Completed. ");
25        sc.close();
26    }
27 }
28 class OrderThread extends Thread{
29     public void run(){
30         System.out.println("Order processing completed. ");
31     }
32 }
33 class PaymentThread extends Thread{
34     public void run(){
```




SIMATS
ENGINEERING

Secure Exit

SIMATS Online Programming Lab

JAVA PROGRAMMING

```
16 pt.start();
17 pt.join();
18 if(pm.equalsIgnoreCase("Failed")){
19     System.out.println("Payment failed , delivery assignment
20 }else{
21     dt.start();
22     dt.join();
23 }
24 System.out.println("Execution Completed. ");
25 sc.close();
26 }
27 }
28 class OrderThread extends Thread{
29     public void run(){
30         System.out.println("Order processing completed. ");
31     }
32 }
33 class PaymentThread extends Thread{
34     public void run(){
35         System.out.println("Payment Processing completed. ");
36     }
37 }
38 class DeliveryThread extends Thread{
39     String agent;
40     DeliveryThread(String agent){
41         this.agent=agent;
42     }
43     public void run(){
44         System.out.println("Delivery assigned to "+ agent);
45     }
46 }
```

Submitted